

Database Security

Database security refers to protection from malicious access. Absolute protection of the database from malicious abuse is not possible, but the cost to the perpetrator can be made high enough to deter most if not all attempts to access the database without proper authority. The data stored in the database need protection from unauthorized access and malicious destruction or alteration, in addition to the protection against accidental introduction of inconsistency that integrity constraints provide.

Needs of security:
To protect the database, we must take security measure at several levels:

Database system; some database system user may be authorized to access only a limited portion of the database. Other user may be allowed to issue queries, but mat be forbidden to modify the data. It is the responsibility of the database system to ensure that these authorization restrictions are not violated.

Operating system; No matter how secure the database system is, weakness in operating system security may as a means of unauthorized access to the database.

Network; since almost all database system allows remote access through terminals or network, software level security within the network software is as important as physical security, both on the internet and in private networks**.**

Physical; Sites with computer system must be physically secured against armed or surreptitious entry by intrudes.

Human; Users must be authorized carefully to reduce the chance of any user giving access to an intruder in exchange for a bribe or other favors. Security is need in database for the following reasons.

1. ***Protection from improper access***- only authorized users should be granted access to objects of DBMS. This control should be applied on smaller objects (record, attribute, value).
2. ***Protection from inference*** - inference of confidential information from available data should be avoided. This regards mainly statistical DBMSs.
3. ***Database integrity*** - partially is ensured with system controls of DBMS (atomic transactions) and various back-up and recovery procedures and partially with security procedures.
4. ***Operational data integrity*** - logical consistence of data during concurrent transactions (concurrency manager), serializability and isolation of transactions (locking techniques).

© Bishwo Prakash Pokharel

5. ***Semantic data integrity*** - ensuring that attribute values are in allowed ranges. This is ensured with integrity constraints.
6. ***Accountability and auditing*** - there should be possibility to log all data accesses.
7. ***User authentication*** - there should be unambiguous identification of each DBMS user. This is basis for all authorization mechanisms.
8. ***Management and protection of sensitive data*** - access should be granted only to narrow round of users.
9. ***Multilevel security*** - data may be classified according to their sensitivity. Access granting should then depend on that classification.
10. ***Confinement (subject isolation)*** - there is necessity to isolate subjects to avoid uncontrolled data flow between programs (memory channels, covert channels).

7.1 Access Control: Discretionary and Mandatory

Access control is responsible for control of rules determined by security policies for all direct accesses to the system. Traditional control systems work with notions *subject*, *object* and *operation*.
For better image look at the figure of secure DBMS.

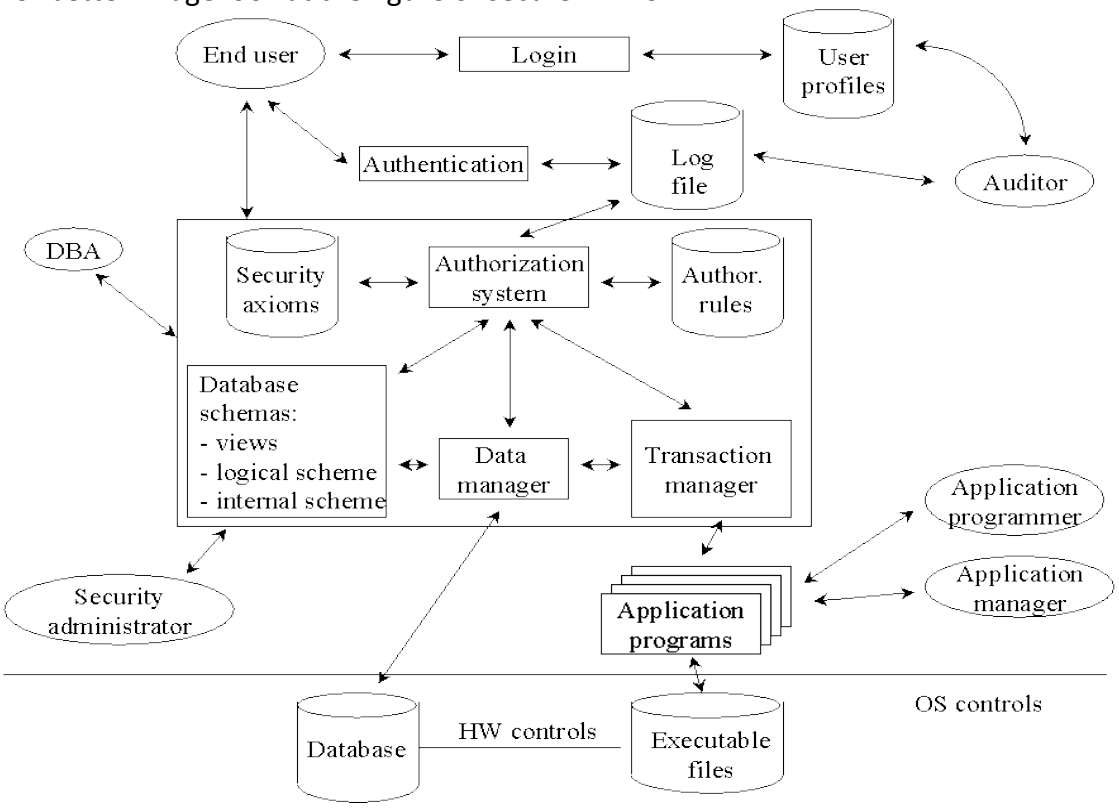


Fig: Schema of secure database management system

The term access control is something of an ambiguous term. To some it could be interpreted as controlling the access to a system from an external source (for example controlling the login process via which users gain access to a server or desktop system). In fact, such access control is actually referred to as Authentication or Identity Verification and is not what is mean by Access Control in this context.

The term Access Control actually refers to the control over access to system resources after a user's account credentials and identity have been authenticated and access to the system resources after a user's account credentials and identity have been authenticated and

access to the system granted. For example, a particular user, or group of users, might only be permitted access to certain files after logging into a system, while simultaneously being denied access to all other resources.

In addition to handling such concern any organization need to address five areas regard to achieving high data quality.

1. **Security policy and disaster recovery:** The organization must establish security policies and make detailed disaster recovery plans. It should determine how the organization is going to maintain a secure system. Disaster recovery will determine how the organization will continue to function should an emergency situation such as flood or fire.
2. **Personnel control:** Monitoring to ensure that the personnel are following established practices, taking regular, vacation, working with other employees, and so forth should be followed. Employee should be trained in those aspects of security and quality that are relevant to their jobs and be encouraged to be aware of and follow standard security and data quality measures.
3. **Physical access control:** Limited access to particular areas within a building is usually a part of controlling physical areas. Sensitive equipment, including hardware and peripherals such as printers can be controlled by placement in the secured areas. Other equipment may be locked to a desk or cabinet, or may have an alarm attached. Backup data tapes must be kept in fireproof data safes or at a safe location.
4. **Maintenance control:** An area of control that helps to maintain data quality but that is often overlooked is maintenance control. Organization should review external maintenance agreements for all hardware and software they are using to ensure that appropriate responses are agreed to for maintaining data quality.
5. **Data protection and privacy:** It refers about the right of individuals to not have personal information collected and misestimated casually have intensified as more of the population has become familiar with computers and as communications among computers have proliferated.

Mandatory Access Control (MAC):

Mandatory Access Control (MAC) is the strictest of all levels of control. The design of MAC was defined, and is primarily used by the government.

MAC takes a hierarchical approach to controlling access to resources. Under a MAC enforced environment access to all resource objects (such as data files) is controlled by settings defined by the system administrator. As such, all access to resource objects is strictly controlled by the operating system based on system administrator configured settings. It is not possible under MAC enforcement for users to change the access control of a resource.

Mandatory Access Control begins with security labels assigned to all resource objects on the system. These security labels contain two pieces of information - a classification (top secret, confidential etc) and a category (which is essentially an indication of the management level, department or project to which the object is available).

© Bishwo Prakash Pokharel

Similarly, each user account on the system also has classification and category properties from the same set of properties applied to the resource objects. When a user attempts to access a resource under Mandatory Access Control the operating system checks the user's classification and categories and compares them to the properties of the object's security label. If the user's credentials match the MAC security label properties of the object access is allowed. It is important to note that both the classification and categories must match. A user with top secret classification, for example, cannot access a resource if they are not also a member of one of the required categories for that object.

Mandatory Access Control is by far the most secure access control environment but does not come without a price. Firstly, MAC requires a considerable amount of planning before it can be effectively implemented. Once implemented it also imposes a high system management overhead due to the need to constantly update object and account labels to accommodate new data, new users and changes in the categorization and classification of existing users.

Discretionary Access Control:
Unlike Mandatory Access Control (MAC) where access to system resources is controlled by the operating system (under the control of a system administrator), Discretionary Access Control (DAC) allows each user to control access to their own data. DAC is typically the default access control mechanism for most desktop operating systems.

Instead of a security label in the case of MAC, each resource object on a DAC based system has an Access Control List (ACL) associated with it. An ACL contains a list of users and groups to which the user has permitted access together with the level of access for each user or group. For example, User A may provide read-only access on one of her files to User B, read and write access on the same file to User C and full control to any user belonging toGroup 1.
It is important to note that under DAC a user can only set access permissions for resources which they already own. A hypothetical User Acannot, therefore, change the access control for a file that is owned by User B. User A can, however, set access permissions on a file that she owns. Under some operating systems it is also possible for the system or network administrator to dictate which permissions users are allowed to set in the ACLs of their resources.

Discretionary Access Control provides a much more flexible environment than Mandatory Access Control but also increases the risk that data will be made accessible to users that should not necessarily be given access.

Role Based Access Control:
Role Based Access Control (RBAC), also known as Non discretionary Access Control, takes more of a real world approach to structuring access control. Access under RBAC is based on a user's job function within the organization to which the computer system belongs.

Essentially, RBAC assigns permissions to particular roles in an organization. Users are then assigned to that particular role. For example, an accountant in a company will be assigned to the Accountant role, gaining access to all the resources permitted for all accountants on the system. Similarly, a software engineer might be assigned to the developer role.

© Bishwo Prakash Pokharel

Roles differ from groups in that while users may belong to multiple groups, a user under RBAC may only be assigned a single role in an organization. Additionally, there is no way to provide individual users additional permissions over and above those available for their role. The accountant described above gets the same permissions as all other accountants, nothing more and nothing less.

Rule Based Access Control:
Rule Based Access Control (RBAC) introduces acronym ambiguity by using the same four letter abbreviation (RBAC) as Role Based Access Control.

Under Rules Based Access Control, access is allowed or denied to resource objects based on a set of rules defined by a system administrator. As with Discretionary Access Control, access properties are stored in Access Control Lists (ACL) associated with each resource object. When a particular account or group attempts to access a resource, the operating system checks the rules contained in the ACL for that object.

Examples of Rules Based Access Control include situations such as permitting access for an account or group to a network connection at certain hours of the day or days of the week.
As with MAC, access control cannot be changed by users. All access permissions are controlled solely by the system administrator.

7.2 Authorization and Authentication

Authentication:
Authentication means verifying the identity of someone (a user, device, or other entity) who wants to use data, resources, or applications. Validating that identity establishes a trust relationship for further interactions. Authentication also enables accountability by making it possible to link access and actions to specific identities. After authentication, authorization processes can allow or limit the levels of access and action permitted to that entity.

For simplicity, the same authentication method is generally used for all database users. To validate the identity of database users and prevent unauthorized use of a database user name, we can authenticate using any combination of the methods as follows:

- **Authentication by Operating System:** Once authenticated by the operating system, users can connect to database more conveniently, without specifying a user name or password. With control over user authentication centralized in the operating system, it need not store or manage user passwords, though it still maintains user names in the database.
- **Authentication by the Network:**
    - Third Party based authentication technologies: If network authentication services are available to us then database can accept authentication from the network service.
    - Public key infrastructure based authentication: Authentication system based on public key cryptography issue digital certificates to user clients, which use them to authenticate directly to servers in the enterprise without directly involving an authentication server.

  - Remote authentication: It supports remote authentication of users through remote dial in user service (RADIUS), a standard lightweight protocol used for user authentication, authorization, and accounting.
- **Authentication by the Oracle Database:** Oracle can authenticate users attempting to connect to a database by using information stored in that database. Database authentication includes the following facilities:
  - Password Encryption: To protect password confidentiality, Oracle always encrypts passwords before sending them over the network. It encrypts the passwords using a modified AES (Advanced Encryption Standard) algorithm.
  - Account locking: Oracle can lock a user's account after a specified number of consecutive failed log-in attempts. You can configure the account to unlock automatically after a specified time interval or to require database administrator intervention to be unlocked. The database administrator can also lock accounts manually, so that they must be unlocked explicitly by the database administrator.
  - Password lifetime and expiration: The database administrator can specify a lifetime for passwords, after which they expire and must be changed before account login is again permitted. A grace period can be established, during which each attempt to login to the database account receives a warning message to change the password. If it is not changed by the end of that period, then the account is locked.
- **Authentication by the secure socket layer protocol:** The secure socket layer (SSL) protocol is an application layer protocol. Users identified either externally or globally (external or global users) can authenticate to a database through SSL.
- **Authentication of Database Administrators:** Database administrators perform special operations (such as shutting down or starting up a database) that should not be performed by normal database users.

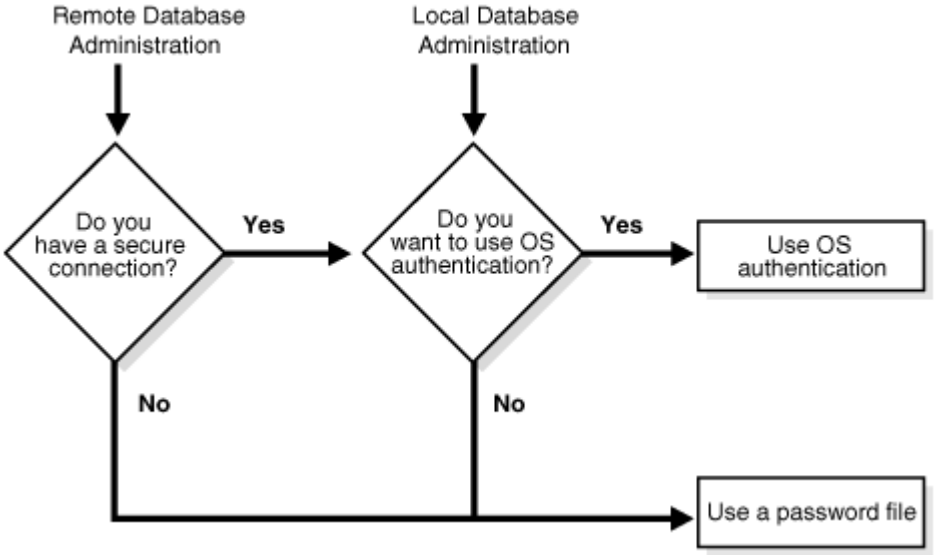


Fig: Database Administrator Authentication Methods

Authorization:

We may assign a user several forms of authorization on parts of the database. For example:

- **Read authorization:** allows reading, but not modification of data.
- **Insert authorization:** allows insertion of new data but not modification of existing data.

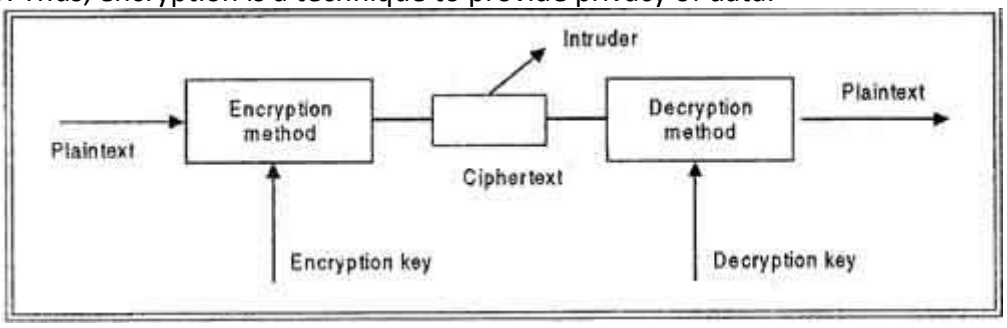
- **Update authorization:** allows modification but not deletion of data.
- **Delete authorization:**allows deletion of data. We may assign the user all, none, or a combination of these type of authorization. In addition to these forms of authorization for access to data. We may grant a user authorization to modify the database schema.
- **Index authorization:** allows the creation and deletion of indices.
- **Resources authorization:** allows the creation of new relations.
- **Alteration authorization:** allows the addition or deletion of attributes in a relation.
- **Drop authorization:** allows the deletion of relations.

The drop and delete authorization differs in that delete authorization allows deletion of tuples only. If a user deletes all tuples of relation, the relation still exists, but it is empty. If the relation is dropped, it no longer exists.
It regulates the ability to create a new relation through resources authorization. A user with resources authorization who creates a new relation is given all privileges on that relation automatically.

Index authorization may appear unnecessary, since the creation or deletion of an index does not alter data in relation. Rather, indices are a structure for performance enhancements. However, indices also consume space, and all database modification are required to updates indices if index authorization were granted to all customer, those who performed updates would be tempted to delete indices, where those who issued queries would be tempted to creates numerous indices, to allows the database administrator to regulates the use of system resources, it is necessary to treat index creation as a privilege.

7.3 Data encryption and decryption

A DBMS can use encryption to protect information in certain situations where the normal security mechanisms of the DBMS are not adequate. For example, an intruder may steal tapes containing some data or tap a communication line. By storing the transmitting data in an encrypted form, the DBMS ensures that such stolen data is not intelligible to the intruder. Thus, encryption is a technique to provide privacy of data.



In encryption, the message to be encrypted is known as plaintext. The plaintext is transformed by a function that is parameterized by a key. The output of the encryption process is known as the cipher text. Ciphertext is then transmitted over the network. The process of converting the plaintext to ciphertext is called as Encryption and process of converting the ciphertext to plaintext is called as Decryption. Encryption is performed at the transmitting end and decryption is performed at the receiving end. For encryption process we need the encryption key and for decryption process we need decryption key as shown in

figure. Without the knowledge of decryption key intruder cannot break the ciphertext to plaintext. This process is also called as Cryptography.

The basic idea behind encryption is to apply an encryption algorithm, which may' be accessible to the intruder, to the original data and a user-specified or DBA-specified encryption key, 'which is kept secret. The output of the algorithm is the encrypted version of the data. There is also a decryption algorithm, which takes the encrypted data and the decryption key as input and then returns the original data. Without the correct decryption key, the decryption algorithm produces gibberish. Encryption and decryption keys may be same or· different but there must be relation between the both which must me secret.

Techniques used for Encryption:
There are following techniques used for encryption process:

- Substitution ciphers
- Transposition ciphers

Substitution Ciphers: In a substitution cipher each letter or group of letters is replaced by another letter or group of letters to mask them For example: a is replaced with D, b with E, c with F and z with C. In this way *attack* becomes *DWWDFN.* The substitution ciphers are not much secure because intruder can easily guess the substitution characters.

Transposition Ciphers: Substitution ciphers preserve the order of the plaintext symbols but mask them-;-The transposition cipher in contrast reorders the letters but do not mask them. For this process a key is used. For example: *iliveinqadian* may be coded as*divienaniqnli.* The transposition ciphers are more secure as compared to substitution ciphers.

Algorithms for Encryption Process
There are commonly used algorithms for encryption process. These are:

• Data Encryption Standard (DES)
• Public Key Encryption

Data Encryption Standard (DES)

It uses both a substitution of characters and a rearrangement of their order on the basis of an encryption key. The main weakness of this approach is that authorized users must be told the encryption key, and the mechanism for communicating this information is vulnerable to clever intruders.

Public Key Encryption

Another approach to encryption, called public-key encryption, has become increasingly popular in recent years. The encryption scheme proposed by Rivest, Shamir, and Adheman, called RSA, is a well-knownexample of public-key encryption. Each authorized user has a public encryption key, known to everyone and a private decryption key (used by the decryption algorithm), chosen by the user and known only to him or her. The encryption and decryption algorithms themselves are assumed to be publicly known.

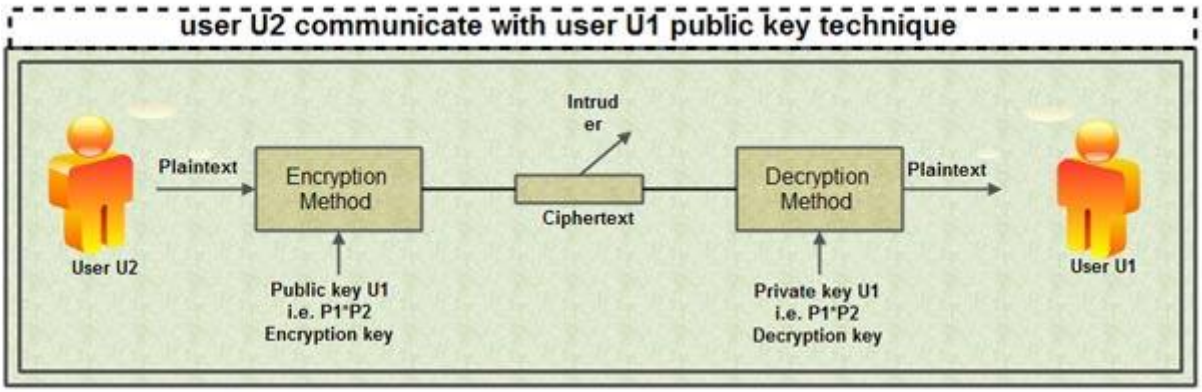
© Bishwo Prakash Pokharel

Consider user called Suneet. Anyone can send Suneet a secret message by encrypting the message using Sunset's publicly known encryption key. Only Suneet can decrypt this secret message because the decryption algorithm required Suneet's decryption key, known only to Suneet. Since users choose their own decryption keys, the weakness 0f DES is avoided.

The main issue for public-key encryption is how encryption and decryption keys are chosen. Technically, public-key encryption algorithms rely on the existence of one-way functions, which are functions whose inverse is computationally very hard to determine.

The RSA algorithm, for example is based on the observation that although checking whether a given number of prime is easy, determining the prime factors of a nonprime number is extremely hard. (Determining the prime factors of a number with over 100 digits can take years of CPU-time on the fastest available computers today.)

We now sketch the intuition behind the RSA algorithm, assuming that the data to be encrypted is an integer 1. To choose an encryption key and a decryption key, our friend Suneet-- create a public key by computing the product of two large prime numbers: PI and P2. The private key consists of the pair (PI, P2) and decryption algorithms cannot be used if the product of PI and P2 is known. So we publish the product PI *P2, but an unauthorized user would need to be able to factor PIP2 to steal data. By choosing PI and P2 to be sufficiently large (over 100 digits), we can make it very difficult (or nearly impossible) for an intruder to factorize it.



Although this technique is secure, but it is also computationally expensive. A hybrid scheme used for secure communication is to use DES keys exchanged via a public-key encryption scheme and DES encryption is used on the data transmitted subsequently.

Disadvantages of encryption

There are following problems of Encryption:

- Key management (i.e. keeping keys secret) is a problem. Even in public-key encryption the decryption key must be kept secret.
- Even in a system that supports encryption, data must often be processed in plaintext form. Thus sensitive data may still be accessible to transaction programs.
- Encrypting data gives rise to serious technical problems at the level of physical storage organization. For example indexing over data, which is stored in encrypted form, can be very difficult.

© Bishwo Prakash Pokharel

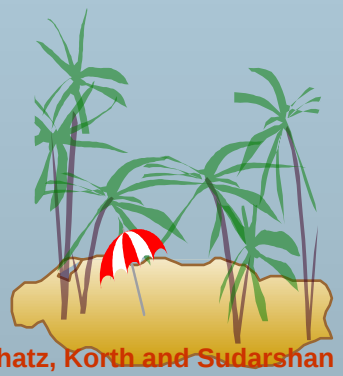
Decryption: Decryption is the process of taking encoded or encrypted text or other data and converting it back into text that you or the computer can read and understand. This term could be used to describe a method of un-encrypting the data manually or with un-encrypting the data using the proper codes or keys.

Data may be encrypted to make it difficult for someone to steal the information. Some companies also encrypt data for general protection of company data and trade secrets. If this data needs to be viewable, it may require decryption. If a decryption passcode or key is not available, special software may be needed to decrypt the data using algorithms to crack the decryption and make the data readable.

© Bishwo Prakash Pokharel

Security

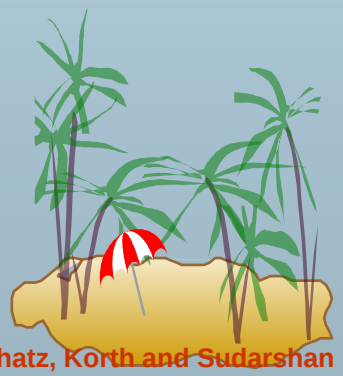
- **Security** - protection from malicious attempts to steal or modify data.
  - ★ Database system level
    - ✔ Authentication and authorization mechanisms to allow specific users access only to required data
    - ✔ We concentrate on authorization in the rest of this chapter
  - ★ Operating system level
    - ✔ Operating system super-users can do anything they want to the database! Good operating system level security is required.
  - ★ Network level: must use encryption to prevent
    - ✔ Eavesdropping (unauthorized reading of messages)
    - ✔ Masquerading (pretending to be an authorized user or sending messages supposedly from authorized users)



©Silberschatz, Korth and Sudarshan

Security (Cont.)

- Physical level
  - Physical access to computers allows destruction of data by intruders; traditional lock-and-key security is needed
  - Computers must also be protected from floods, fire, etc.
    - More in Chapter 17 (Recovery)
- Human level
  - Users must be screened to ensure that an authorized users do not give access to intruders
  - Users should be trained on password selection and secrecy

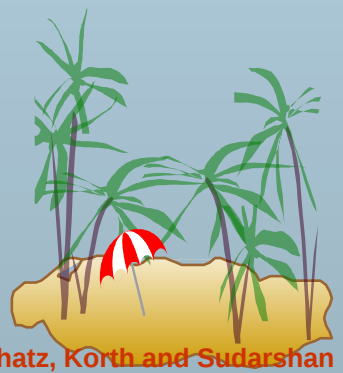


©Silberschatz, Korth and Sudarshan

Authorization

Forms of authorization on parts of the database:

- **Read authorization** - allows reading, but not modification of data.
- **Insert authorization** - allows insertion of new data, but not modification of existing data.
- **Update authorization** - allows modification, but not deletion of data.
- **Delete authorization** - allows deletion of data

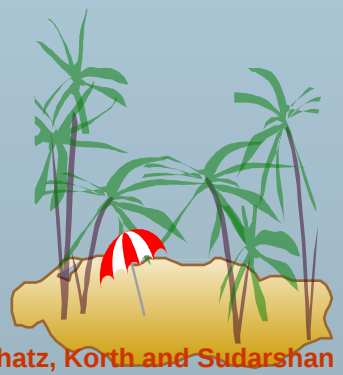


©Silberschatz, Korth and Sudarshan

Authorization (Cont.)

Forms of authorization to modify the database schema:

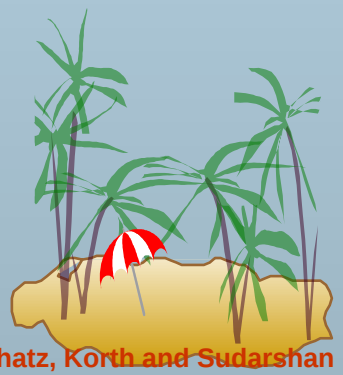
- **Index authorization** - allows creation and deletion of indices.
- **Resources authorization** - allows creation of new relations.
- **Alteration authorization** - allows addition or deletion of attributes in a relation.
- **Drop authorization** - allows deletion of relations.



©Silberschatz, Korth and Sudarshan

Authorization and Views

- Users can be given authorization on views, without being given any authorization on the relations used in the view definition
- Ability of views to hide data serves both to simplify usage of the system and to enhance security by allowing users access only to data they need for their job
- A combination or relational-level security and view-level security can be used to limit a user's access to precisely the data that user needs.

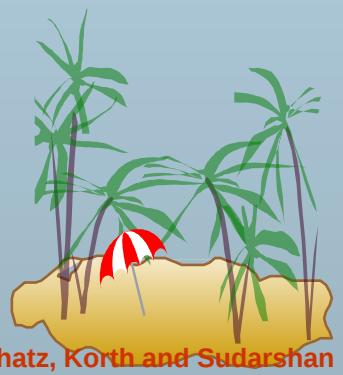


©Silberschatz, Korth and Sudarshan

View Example

- Suppose a bank clerk needs to know the names of the customers of each branch, but is not authorized to see specific loan information.
  - ★ Approach: Deny direct access to the *loan* relation, but grant access to the view *cust-loan*, which consists only of the names of customers and the branches at which they have a loan.
  - ★ The *cust-loan* view is defined in SQL as follows:

```
create view cust-loan as
    select branchname, customer-name
    from   borrower, loan
    where borrower.loan-number = loan.loan-number
```



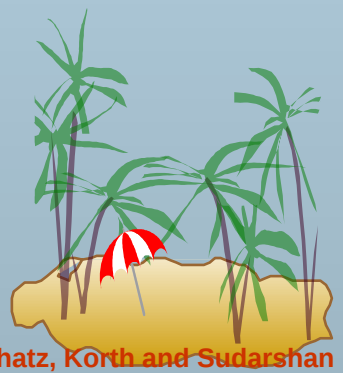
©Silberschatz, Korth and Sudarshan

View Example (Cont.)

- The clerk is authorized to see the result of the query:

  select *
  from *cust-loan*

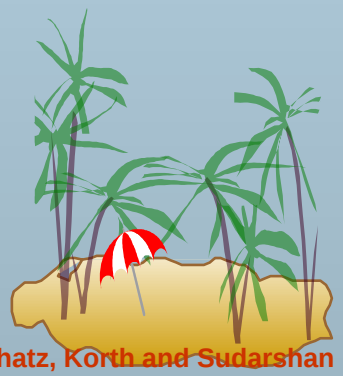
- When the query processor translates the result into a query on the actual relations in the database, we obtain a query on *borrower* and *loan*.
- Authorization must be checked on the clerk's query before query processing begins.



©Silberschatz, Korth and Sudarshan

Authorization on Views

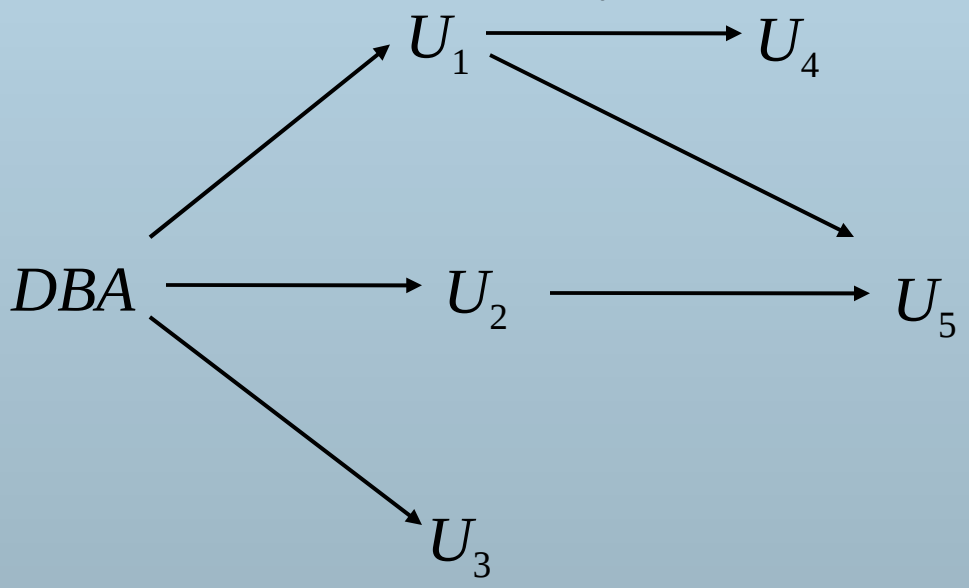
- Creation of view does not require **resources** authorization since no real relation is being created
- The creator of a view gets only those privileges that provide no additional authorization beyond that he already had.
- E.g. if creator of view *cust-loan* had only **read** authorization on *borrower* and *loan*, he gets only **read** authorization on *cust-loan*



©Silberschatz, Korth and Sudarshan

Granting of Privileges

- The passage of authorization from one user to another may be represented by an authorization graph.
- The nodes of this graph are the users.
- The root of the graph is the database administrator.
- Consider graph for update authorization on loan.
- An edge $U_i \rightarrow U_j$ indicates that user U_i has granted update authorization on loan to U_j.



©Silberschatz, Korth and Sudarshan

Authorization Grant Graph

- *Requirement*: All edges in an authorization graph must be part of some path originating with the database administrator
- If DBA revokes grant from U_1:
  - ★ Grant must be revoked from U_4 since U_1 no longer has authorization
  - ★ Grant must not be revoked from U_5 since U_5 has another authorization path from DBA through U_2
- Must prevent cycles of grants with no path from the root:
  - ★ DBA grants authorization to U_7
  - ★ U7 grants authorization to U_8
  - ★ U8 grants authorization to U_7
  - ★ DBA revokes authorization from U_7
- Must revoke grant U_7 to U_8 and from U_8 to U_7 since there is no path from DBA to U_7 or to U_8 anymore.



©Silberschatz, Korth and Sudarshan

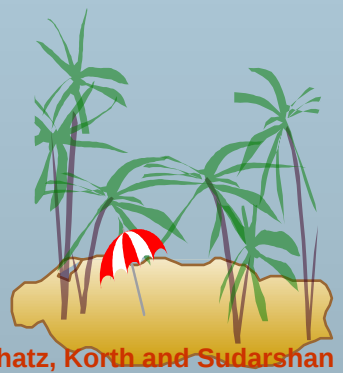
Security Specification in SQL

- The grant statement is used to confer authorization

  grant <privilege list>

  on <relation name or view name> to <user list>

- <user list> is:
  - a user-id
  - *public*, which allows all valid users the privilege granted
  - A role (more on this later)
- Granting a privilege on a view does not imply granting any privileges on the underlying relations.
- The grantor of the privilege must already hold the privilege on the specified item (or be the database administrator).

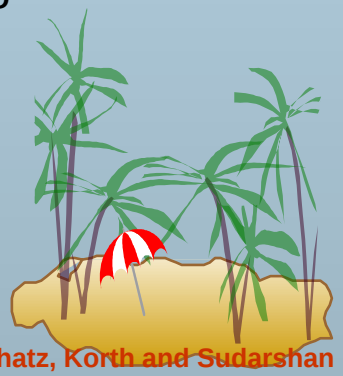


©Silberschatz, Korth and Sudarshan

Privileges in SQL

- **select:** allows read access to relation,or the ability to query using the view
  - ★ Example: grant users U_1, U_2, and U_3 **select** authorization on the *branch* relation:

    grant select on *branch* **to** U_1, U_2, U_3
- **insert**: the ability to insert tuples
- **update**: the ability to update using the SQL update statement
- **delete**: the ability to delete tuples.
- **references**: ability to declare foreign keys when creating relations.
- **usage**: In SQL-92; authorizes a user to use a specified domain
- **all privileges**: used as a short form for all the allowable privileges



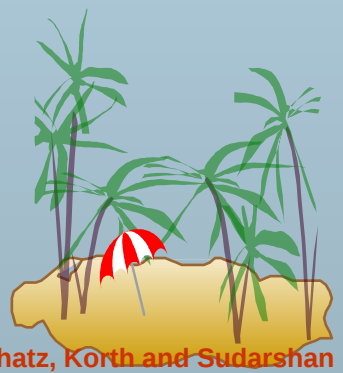
©Silberschatz, Korth and Sudarshan

Privilege To Grant Privileges

- **with grant option**: allows a user who is granted a privilege to pass the privilege on to other users.
  - ★ Example:

grant select on *branch* **to** U_1 **with grant option**

gives U_1 the **select** privileges on branch and allows U_1 to grant this privilege to others



©Silberschatz, Korth and Sudarshan

Roles

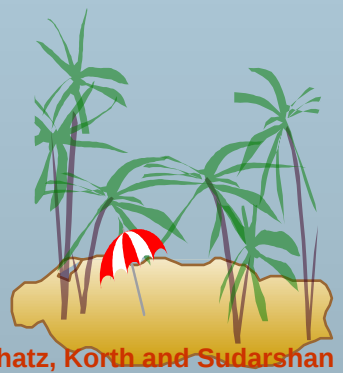
- Roles permit common privileges for a class of users can be specified just once by creating a corresponding “role”
- Privileges can be granted to or revoked from roles, just like user
- Roles can be assigned to users, and even to other roles
- SQL:1999 supports roles

create role *teller*
create role *manager*

grant select on *branch* **to** *teller*
grant update (*balance***) on** *account* **to** *teller*
grant all privileges on *account* **to** *manager*

grant *teller* **to** *manager*

grant *teller* **to** *alice, bob*
grant *manager* **to** *avi*



©Silberschatz, Korth and Sudarshan

Revoking Authorization in SQL

- The **revoke** statement is used to revoke authorization.

revoke<privilege list>

on <relation name or view name> **from** <user list> [**restrict**|**cascade**]

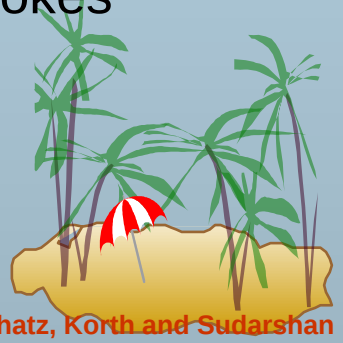
- Example:

revoke select on *branch* **from** U_1, U_2, U_3 **cascade**

- Revocation of a privilege from a user may cause other users also to lose that privilege; referred to as cascading of the **revoke**.
- We can prevent cascading by specifying **restrict**:

  revoke select on *branch* **from** U_1, U_2, U_3 **restrict**

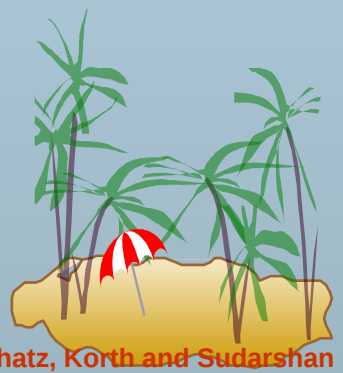
  With **restrict**, the **revoke** command fails if cascading revokes are required.



©Silberschatz, Korth and Sudarshan

Revoking Authorization in SQL (Cont.)

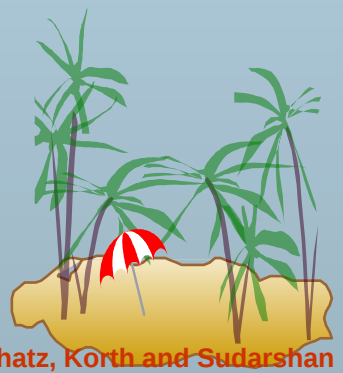
- <privilege-list> may be **all to** revoke all privileges the revokee may hold.
- If <revokee-list> includes **public** all users lose the privilege except those granted it explicitly.
- If the same privilege was granted twice to the same user by different grantees, the user may retain the privilege after the revocation.
- All privileges that depend on the privilege being revoked are also revoked.



©Silberschatz, Korth and Sudarshan

Limitations of SQL Authorization

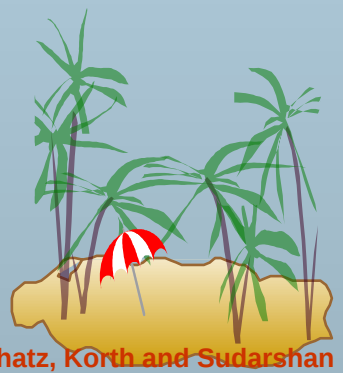
- SQL does not support authorization at a tuple level
  - ★ E.g. we cannot restrict students to see only (the tuples storing) their own grades
- All end-users of an application (such as a web application) may be mapped to a single database user
- The task of authorization in above cases falls on the application program, with no support from SQL
  - ★ Authorization must be done in application code, and may be dispersed all over an application
  - ★ Checking for absence of authorization loopholes becomes very difficult since it requires reading large amounts of application code



©Silberschatz, Korth and Sudarshan

Encryption

- Data may be *encrypted* when database authorization provisions do not offer sufficient protection.
- Properties of good encryption technique:
  - ★ Relatively simple for authorized users to encrypt and decrypt data.
  - ★ Encryption scheme depends not on the secrecy of the algorithm but on the secrecy of a parameter of the algorithm called the encryption key.
  - ★ Extremely difficult for an intruder to determine the encryption key.



©Silberschatz, Korth and Sudarshan

Encryption (Cont.)

- *Data Encryption Standard* (DES) substitutes characters and rearranges their order on the basis of an encryption key which is provided to authorized users via a secure mechanism. Scheme is no more secure than the key transmission mechanism since the key has to be shared.
- Advanced Encryption Standard (AES) is a new standard replacing DES, and is based on the Rijndael algorithm, but is also dependent on shared secret keys
- *Public-key encryption* is based on each user having two keys:
  - ★ *public key* – publicly published key used to encrypt data, but cannot be used to decrypt data
  - ★ *private key* -- key known only to individual user, and used to decrypt data. Need not be transmitted to the site doing encryption.

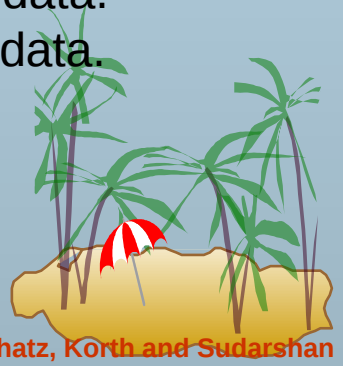
  Encryption scheme is such that it is impossible or extremely hard to decrypt data given only the public key.
- The RSA public-key encryption scheme is based on the hardness of factoring a very large number (100's of digits) into its prime components.



©Silberschatz, Korth and Sudarshan

Authentication

- Password based authentication is widely used, but is susceptible to sniffing on a network
- **Challenge-response** systems avoid transmission of passwords
  - ★ DB sends a (randomly generated) challenge string to user
  - ★ User encrypts string and returns result.
  - ★ DB verifies identity by decrypting result
  - ★ Can use public-key encryption system by DB sending a message encrypted using user's public key, and user decrypting and sending the message back
- **Digital signatures** are used to verify authenticity of data
  - ★ E.g. use private key (in reverse) to encrypt data, and anyone can verify authenticity by using public key (in reverse) to decrypt data. Only holder of private key could have created the encrypted data.
  - ★ Digital signatures also help ensure **nonrepudiation:** sender cannot later claim to have not created the data



©Silberschatz, Korth and Sudarshan